



MAPSTRUCT WITH SPRING BOOT



CHEAT SHEET (Production Ready)

Type-safe • Fast • Clean Code • Production Ready

MapStruct is a code generator that simplifies object mapping in Java.
It generates implementation at compile time – no reflection, highest performance.



WHY MAPSTRUCT?

- ✓ Compile-time type safety
- ✓ High performance (No reflection)
- ✓ Less boilerplate code
- ✓ Easy to test
- ✓ IDE friendly
- ✓ Works great with Spring Boot



WHEN TO USE?

- Entity ⇌ DTO mapping
- Request ⇌ Response mapping
- Microservices & APIs
- Large projects (to avoid manual mapping)
- Complex nested object mapping



MAPSTRUCT IN SPRING BOOT (QUICK SETUP)

1. Add dependency

```
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct</artifactId>
  <version>1.5.5.Final</version>
</dependency>
```

2. Add annotation processor (Maven)

```
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct-processor</artifactId>
  <version>1.5.5.Final</version>
  <scope>provided</scope>
</dependency>
```

3. Enable in Mapper

```
@Mapper
public interface UserMapper {
    UserDto toDto(User user);
    User toEntity(UserDto dto);
}
```

4. Use in Spring Boot

```
@Service
public class UserService {
    private final UserMapper mapper;

    public UserService(UserMapper mapper) {
        this.mapper = mapper;
    }
}
```

1 BASIC MAPPING EXAMPLES

1. Simple Mapping

```
@Mapper
public interface UserMapper {
    UserDto toDto(User user);
    User toEntity(UserDto dto);
}
```

2. Different Field Names

```
@Mapper
public interface UserMapper {
    @Mapping(source = "fullName", target = "name")
    @Mapping(source = "emailId", target = "email")
    UserDto toDto(User user);
}
```

3. Ignore Field

```
@Mapper
public interface UserMapper {
    @Mapping(target = "password", ignore = true)
    UserDto toDto(User user);
}
```

4. Constant Value

```
@Mapper
public interface UserMapper {
    @Mapping(target = "status", constant = "ACTIVE")
    UserDto toDto(User user);
}
```

5. Default Expression

```
@Mapper
public interface UserMapper {
    @Mapping(target = "createdAt",
        expression = "java.time.LocalDateTime.now()")
    UserDto toDto(User user);
}
```

6. Nested Mapping

```
@Mapper
public interface OrderMapper {
    OrderDto toDto(Order order);
}

@Mapper
public interface CustomerMapper {
    CustomerDto toDto(Customer customer);
}
```

MapStruct will automatically use CustomerMapper for nested Customer object.

★ Key Takeaway

- ✔ MapStruct generates implementation at compile time.
- ✔ It is fast, type-safe and easy to maintain.
- ✔ Use `@Mapping` for custom logic.

2 ADVANCED MAPPING

1. Collection Mapping

```
@Mapper
public interface UserMapper {
    List<UserDto> toDtoList(List<User> users);
    Set<User> toEntitySet(Set<UserDto> dtos);
}
```

2. Update Existing Object (Partial Update)

```
@Mapper
public interface UserMapper {
    @Mapping(target = "id", ignore = true)
    void updateUserFromDto(UserDto dto,
        @MappingTarget User user);
}
```

✔ @MappingTarget updates the existing object.

3. Map with Custom Method

```
@Mapper
public interface ProductMapper {
    @Mapping(source = "priceInCents", target = "price")
    ProductDto toDto(Product product);

    default BigDecimal toPrice(Integer priceInCents) {
        if (priceInCents == null) return null;
        return new BigDecimal(priceInCents).divide(
            new BigDecimal(100));
    }
}
```

4. Multiple Source Objects

```
@Mapper
public interface UserMapper {
    @Mapping(source = "user.id", target = "userId")
    @Mapping(source = "address.city", target = "city")
    UserResponse toResponse(User user, Address address);
}
```

5. Inherit Configuration

```
@MapperConfig
public interface CentralConfig {
    @Mapping(target = "createdAt",
        expression = "java(java.time.LocalDateTime.now())")
}

@Mapper(config = CentralConfig.class)
public interface OrderMapper {
    OrderDto toDto(Order order);
}
```

📌 Useful Annotations

- ✔ @Mapper - Marks the interface as mapper
- ✔ @Mapping - Custom field mapping
- ✔ @Mappings - Multiple mappings
- ✔ @MappingTarget - Update existing object
- ✔ @Context - Pass context or services
- ✔ @MapperConfig - Reusable configuration
- ✔ @EnumMapping - Enum mapping strategies

1. Component Model

Use `componentModel = "spring"` to make mappers Spring Beans.

```
@Mapper(componentModel = "spring")
public interface UserMapper {
    UserDto toDto(User user);
}
```

Now you can `@Autowired` or constructor inject the mapper.

2. Injection Strategy

Choose how dependencies are injected.

```
@Mapper(componentModel = "spring",
        injectionStrategy = InjectionStrategy.CONSTRUCTOR)
public interface UserMapper {
}
```

InjectionStrategy options:

- FIELD (default)
- CONSTRUCTOR (recommended)
- SETTER

3. Unmapped Target Policy

Fail build if any target field is not mapped.

```
@Mapper(componentModel = "spring",
        unmappedTargetPolicy = ReportingPolicy.ERROR)
public interface UserMapper {
}
```

Other options:

- IGNORE (default)
- WARN
- ERROR

4. Null Value Handling

a) Null Value Property Mapping

```
@Mapper(
    nullValuePropertyMappingStrategy =
        NullValuePropertyMappingStrategy.IGNORE)
public interface UserMapper {
    void updateUser(UserDto dto, @MappingTarget User user);
}
```

IGNORE -> null values in source will not overwrite target values.

b) Null Value Check Strategy

```
@Mapper(nullValueCheckStrategy =
        NullValueCheckStrategy.ALWAYS)
public interface UserMapper {
}
```

ALWAYS -> generates null checks for all source properties.

5. Date / Time Format

```
@Mapper
public interface EventMapper {
    @Mapping(target = "date",
            dateFormat = "dd-MM-yyyy")
    EventDto toDto(Event event);
}
```



REMEMBER

Good mapping is not about magic,
it's about clarity and correctness!

1. Layered Mapping

- Entity → DTO → Response
- Request → DTO → Entity



Entity



DTO



Response / Request

2. Mapper Composition

Use other mappers inside a mapper.

```
@Mapper(uses = {AddressMapper.class})
public interface UserMapper {
    UserDto toDto(User user);
}
```

Helps in modular and reusable mapping.

3. Avoid Business Logic

- Keep mappers simple.
- Don't add business logic.
- Use service layer for business rules.

4. Keep Mappers Small

- Split large mappers into multiple small mappers.
- Improves readability and maintainability.



5. Test Your Mappers

```
@ExtendWith(MockitoExtension.class)
class UserMapperTest {
    private final UserMapper mapper =
        Mappers.getMapper(UserMapper.class);

    @Test
    void testToDto() {
        User user = new User();
        user.setId(1L);
        user.setName("John");
        UserDto dto = mapper.toDto(user);
        assertEquals("John", dto.getName());
    }
}
```

Best Practices Checklist

- ✓ Use constructor injection
- ✓ Keep mappers focused and small
- ✓ Use @MapperConfig for common settings
- ✓ Enable unmappedTargetPolicy = ERROR
- ✓ Write unit tests for critical mappings

Common Pitfalls

- ✗ Adding business logic in mapper
- ✗ Ignoring null handling
- ✗ Not testing mappers
- ✗ Large monolithic mappers
- ✗ Forgetting to update mappings after model changes



1. Entities

```
@Data
class User {
    private Long id;
    private String fullName;
    private String email;
    private LocalDateTime createdAt;
    private Address address;
}
```

```
@Data
class Address {
    private String city;
    private String country;
}
```

2. DTOs

```
@Data
class UserDto {
    private Long id;
    private String name;
    private String emailId;
    private String city;
}
```

3. Mapper

```
@Mapper(componentModel = "spring", uses = {})
public interface UserMapper {
    @Mapping(source = "fullName", target = "name")
    @Mapping(source = "email", target = "emailId")
    @Mapping(source = "address.city", target = "city")
    UserDto toDto(User user);
}
```

4. Usage in Service

```
@Service
public class UserService {
    private final UserRepository repo;
    private final UserMapper mapper;

    public UserDto getUser(Long id) {
        User user = repo.findById(id).orElseThrow();
        return mapper.toDto(user);
    }
}
```

MapStruct itself has no runtime properties.
But you can configure related compiler behavior.

application.properties / application.yml

```
# No runtime properties required for MapStruct
# But ensure the following compiler args (Maven/Gradle)
# are set for better experience
```

Maven Compiler Plugin (Recommended)

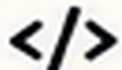
```
<configuration>
  <compilerArgs>
    <arg>-Amapstruct.defaultComponentModel=spring</arg>
    <arg>-Amapstruct.unmappedTargetPolicy=ERROR</arg>
    <arg>-Amapstruct.suppressGeneratorTimestamp=true</arg>
  </compilerArgs>
</configuration>
```

Explanation

- defaultComponentModel=spring -> Default Spring component model.
- unmappedTargetPolicy=ERROR -> Fail build if any target field is not mapped.
- suppressGeneratorTimestamp=true -> Cleaner git diffs.

★ Key Takeaway

MapStruct + Spring Boot =
Clean, Fast, Type-safe and Maintainable Code.
Use it to write less boilerplate and more business value!



SAMPLE JSON FILE TRANSFORMATION

1 INPUT JSON FILE

```
// user-input.json
{
  "fullName": "Aisha Khan",
  "email": "aisha@example.com",
  "address": {
    "city": "Kolkata",
    "country": "IN"
  }
}
```

2 JAVA REQUEST / DOMAIN / RESPONSE

```
record AddressRequest(
    String city,
    String country) {}

record CreateUserRequest(
    String fullName,
    String email,
    AddressRequest address) {}

class User {
    String displayName;
    String email;
    String city;
    String status;
}
```

3 MAPSTRUCT MAPPER

```
@Mapper(
    componentModel = "spring",
    unmappedTargetPolicy =
        ReportingPolicy.ERROR)
interface UserMapper {

    @Mapping(target="displayName",
        source="fullName")
    @Mapping(target="city",
        source="address.city")
    @Mapping(target="status",
        constant="ACTIVE")
    User toEntity(
        CreateUserRequest source);

    @Mapping(target="name",
        source="displayName")
    @Mapping(target="location",
        source="city")
    UserResponse toResponse(User source);
}
```

4 READ, MAP AND WRITE

```
CreateUserRequest request =
    objectMapper.readValue(
        resource.getInputStream(),
        CreateUserRequest.class);

User user = mapper.toEntity(request);

UserResponse response =
    mapper.toResponse(user);

String result =
    objectMapper
        .writerWithDefaultPrettyPrinter()
        .writeValueAsString(response);
```

5 OUTPUT JSON

```
{
  "name": "Aisha Khan",
  "email": "aisha@example.com",
  "location": "Kolkata",
  "status": "ACTIVE"
}
```

IMPORTANT Jackson reads and writes JSON files.

MapStruct maps Java objects using generated compile-time code.

Flow: JSON file -> Jackson DTO -> MapStruct entity -> MapStruct response -> Jackson JSON

Production rule: keep parsing, mapping and business logic as separate responsibilities.